CSE4203: Principles of Software Architecture Dr. Nada Mahmoud Hassan
TA: Eng. Mohamed Fareh
Arab Academy for Science,
Technology & Maritime Transport

Faculty of Computers and Information Technology

Name	ID
Jana Mohamed Samir	231006613
Ahmed Fawzy Alyasergy	231005915
Nour Ahmed Maatouk	231014666
Reem Hussien	231004206

Nexus: Integrated Logistics and Fulfillment Management System

Milestone-1

1. System Overview

Nexus is an enterprise logistics and fulfillment system designed to connect and coordinate the main operations of a retail business. It addresses the problem of managing inventory, product pricing, customer orders, suppliers, and shipments through separate systems, which can lead to inconsistent data, delayed decisions, and difficulties coordinating business operations.

The system provides a centralized platform that synchronizes product information using Spring Boot REST APIs and Firebase Cloud Firestore. It consists of four integrated modules: Inventory Management, Pricing and Promotions, Order Management, and Shipment Tracking and Supplier Synchronization.

The Inventory Management module tracks stock levels across different branches, generates alerts when products reach critical stock thresholds, and supports restocking requests. The Pricing and Promotions module calculates product prices according to different pricing strategies and optional services. The Order Management module supports normal, bulk, and subscription orders, including checkout, cancellation, and order history. Finally, the Shipment Tracking and Supplier Synchronization module coordinates domestic and international deliveries, integrates with legacy supplier systems, and updates inventory when restocking is completed.

The primary users of Nexus are branch managers, global managers, and operational staff responsible for inventory, orders, pricing, and shipments. Suppliers also interact with the system through supplier integration workflows, while customers benefit from more organized order fulfillment.

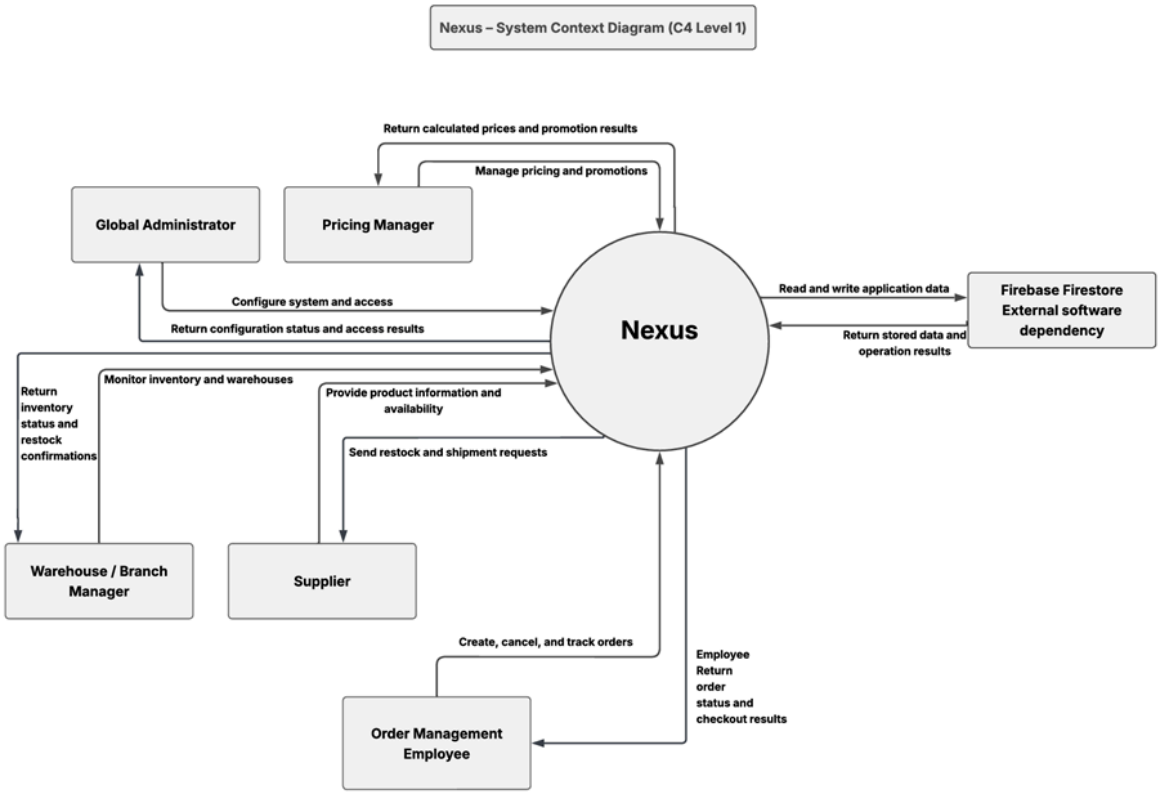
Nexus is architecturally interesting because it combines multiple business modules through shared data and clearly defined interfaces. It uses several software design patterns, including Singleton, Observer, Strategy, Decorator, Factory, Command, Adapter, and Bridge. These patterns help separate responsibilities, reduce coupling, and make it easier to extend the system with new pricing strategies, order types, shipping methods, and integrations. The architecture also introduces important design considerations, such as maintaining data consistency, preventing invalid stock and order operations, and balancing reliable external integrations with system responsiveness.

Overall, Nexus aims to provide a coordinated, maintainable, and extensible logistics platform that improves operational visibility and simplifies the movement of products from suppliers and warehouses to customers.

2. Stakeholders and concerns:

Stakeholder	Key Concerns
Order Manager	Accurate order processing, correct checkout totals, order status tracking, valid cancellations, and access to order history.
Pricing Manager	Accurate price calculations, correct application of discounts, flexible pricing strategies, and proper calculation of optional service fees.
Shipping Manager	Correct shipping method selection, accurate shipment records, reliable supplier communication, shipment tracking, and inventory updates after restocking.
Branch Managers	Accurate stock quantities, timely low-stock alerts, easy dashboard access, and fast restock approvals.
Global Manager (M00)	Correct request information, efficient decision-making, visibility across branches, and timely notifications.
Supplier	Reliable communication, correct supplier data, valid restock requests, and consistent shipment information.
Business Owner	Operational efficiency, reduced stock shortages, accurate pricing, scalability, and maintainability.
Developers / Maintenance Team	Modifiability, maintainability, extensibility, and low coupling

3. System context



4. Functional requirements

ID	User Story / Requirement	Actor(s)	MoSCoW Priority	Prototype Scope (Milestone 3)
FR -01	Branch Authentication & Navigation: As a Branch Manager, I want to securely log into my branch hub (e.g., Cairo, Alexandria, Giza) so that I can manage localized inventory dataset views.	Branch Manager	Must Have	Implemented
FR -02	Real-Time Inventory Traversal: As a Branch Manager, I want to browse products synchronized with cloud database records to monitor live stock quantities and reorder thresholds.	Branch Manager / System	Must Have	Implemented

FR-03	Automated Low-Stock Alert Broadcasting: As the System, I want to monitor product stock levels and broadcast real-time alert notifications when stock drops below threshold.	Nexus System	Must Have	Implemented
FR-04	Restock Request & Approval Workflow: As a Branch Manager, I want to submit restock requests to the Global Manager (M00) and receive live confirmation logs on my dashboard.	Branch Manager / Global Manager (M00)	Must Have	Implemented
FR-05	Dynamic Pricing Calculation: As a Pricing Manager, I want to apply promotional pricing strategies (Normal, VIP, Black Friday) and dynamic decorators (Gift Wrap, Insurance) to items.	Pricing Manager	Must Have	Implemented
FR-06	Multi-Model Order Creation: As an Order Manager, I want to construct Normal, Bulk, and Subscription customer orders using specialized pricing strategies.	Order Manager	Must Have	Implemented
FR-07	Order Checkout & Lifecycle Validation: As an Order Manager, I want to execute order checkouts while validating lifecycle rules to prevent checkout of cancelled orders.	Order Manager / System	Must Have	Implemented

FR-08	Safe Command-Based Order Cancellation: As an Order Manager, I want to safely cancel active orders using command execution before checkout finalization.	Order Manager	Should Have	Implemented
FR-09	Geographic Shipping Method Selection: As a Logistics Manager, I want the system to dynamically resolve domestic vs. international dispatch methods and issue tracking IDs.	Shipping Manager	Must Have	Implemented
FR-10	Legacy Supplier Data Synchronization: As a Supplier Integration System, I want to normalize legacy vendor payload formats into internal entities via an adapter layer.	Supplier System / Logistics Manager	Must Have	Implemented
FR-11	Restock Fulfillment & Stock Updates: As the System, I want to update inventory quantities in Firestore upon confirming supplier fulfillment of restock requests.	Nexus System / Supplier	Should Have	Implemented
FR-12	Automated Multi-Carrier Rate Comparison: As a Shipping Manager, I want to compare live third-party logistics carrier rates side-by-side during dispatch.	Shipping Manager	Could Have	Deferred (Not in M3)

5. Quality attribute scenarios

Scenario 1: Performance

- **Source:** 20 concurrent branch managers and administrators.
- **Stimulus:** Request stock traversal or trigger a pricing calculation query simultaneously under normal operation.
- **Artifact:** User interface and Spring Boot backend REST API / Inventory & Pricing modules.
- **Environment:** Normal operational load.
- **Response:** The system processes the request, retrieves cloud data, computes prices/stock, and renders the data on the dashboard.
- **Response Measure:** 95% of requests complete with a response time of less than 1 second.
-

Scenario 2: Availability

- **Source:** External Cloud Database (Firestore / Firebase Service).
- **Stimulus:** Network interruption or temporary connection timeout occurs during a transaction processing request.
- **Artifact:** FirebaseService connection layer and InventoryService transaction handler.
- **Environment:** Normal operation / cloud connectivity degradation.
- **Response:** The system detects the fault via connection monitoring, logs the error, and throws a protected exception to prevent data corruption or negative stock.
- **Response Measure:** Fault is detected and logged within 3 seconds, ensuring 0 corrupted or negative inventory records.
-

Scenario 3: Security

- **Source:** Unauthorized external user or unauthenticated actor.
- **Stimulus:** Attempts to access restricted global manager endpoints (M00) or modify active order lifecycle states directly via API.
- **Artifact:** Session management and Spring Boot Controller security routing layer.
- **Environment:** Normal operational environment / external threat scenario.
- **Response:** The system authenticates the request, identifies insufficient privileges, blocks access, and logs the security event.
- **Response Measure:** 100% of unauthorized access attempts are blocked instantly, with zero unauthorized state modifications.

Scenario 4: Modifiability

- **Source:** Software Developer.
- **Stimulus:** A request arrives to introduce a new promotional pricing strategy or an additional order model (e.g., Seasonal Flash-Sale Pricing).
- **Artifact:** Pricing Engine module (PricingStrategy interface and concrete implementations).
- **Environment:** Design time / code maintenance phase.
- **Response:** The developer implements the new strategy by adding a new class implementing the interface without modifying existing core pricing or order logic.
- **Response Measure:** The new strategy is integrated, code-reviewed, and unit-tested within 1 person-day with zero regressions in existing modules.

Scenario 5: Testability

- **Source:** QA / Test Engineer.
- **Stimulus:** An automated integration test suite is executed after a new build increment.
- **Artifact:** Order Management module (OrderFactory and CancelOrderCommand).
- **Environment:** Staging / Test environment.
- **Response:** The system executes order creation and safe cancellation workflows, exposing internal object states through logs.
- **Response Measure:** Faults are localized and test suite execution completes in under 2 minutes, achieving at least 90% path coverage.

6. Constraints

· **Technical Constraint :** (Database): The system must use Firebase Cloud (Firestore) as the main cloud database for global inventory.

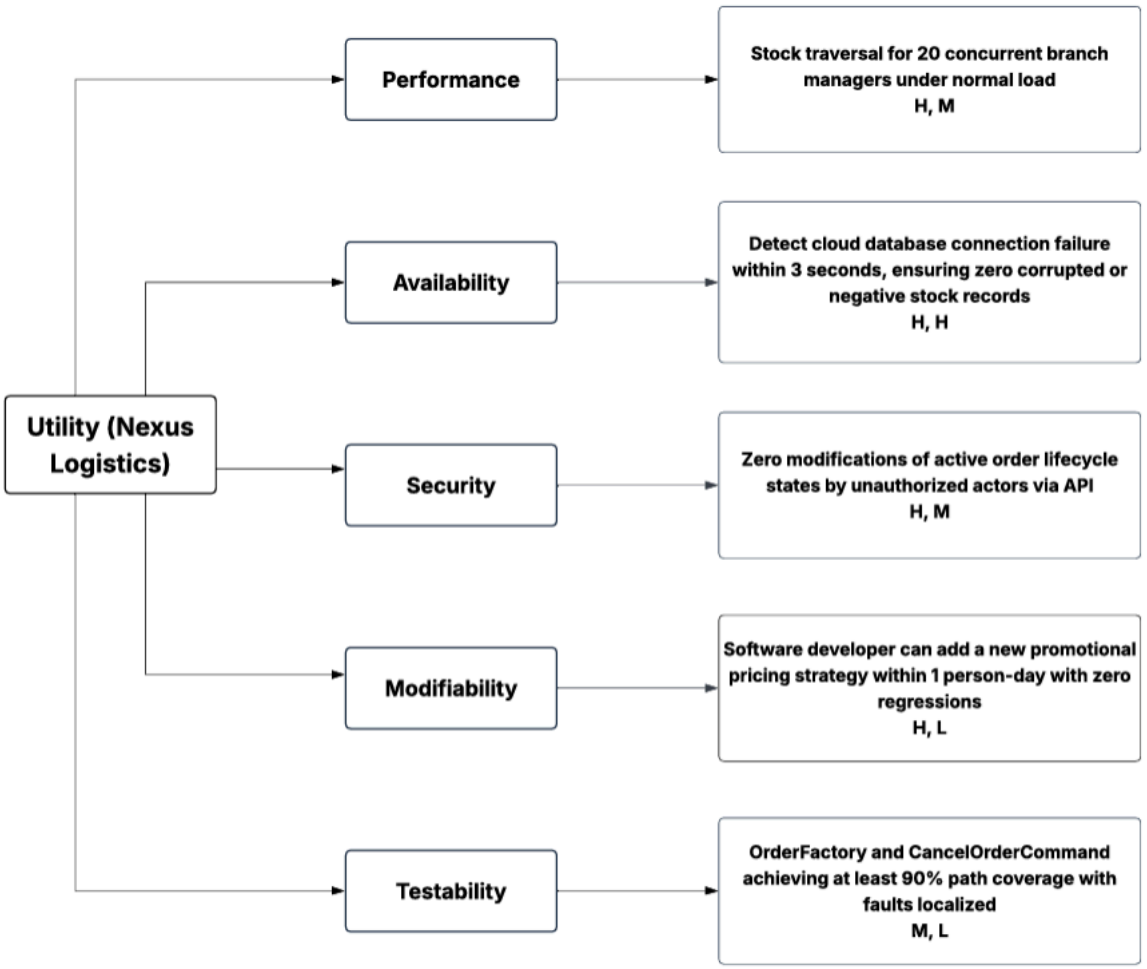
· **Technical Constraint :** (Backend/Frontend): The backend must be built using Spring Boot with REST APIs connected to the frontend.

· **Business Constraint :**(Shipment Confirmation): Shipment updates must wait for confirmation from external suppliers before showing status changes to users.

· **Technical Constraint :** (Legacy Integration): The shipping adapter acts as a translator, allowing the system to communicate with legacy supplier APIs without requiring any changes to the core business logic.

· **Organizational Constraint:** (Team & Schedule): The project must be completed by a 4-student team within the 12-week schedule, keeping the scope focused on core logistics.

7. Architecturally significant requirements (ASRs)



Participants Table:

Task No.	Task Name	Description / Deliverable	Team Member(s)
1	System Overview	One-page description of proposed system, problem solved, user roles, and architectural significance.	**Jana Samir**
2	Stakeholders and Concerns	Table of key stakeholders (users, admins, developers, operators, regulators) and their primary concerns.	**Jana Samir**
3	System Context	C4 Model Level 1 Context Diagram showing system boundaries, actors, and external system interactions.	**Ahmed Alyasergy**
4	Functional Requirements	Eliciting main use cases/stories (8–15), MoSCoW prioritization, and prototype implementation flags.	**Reem Hussein**
5	Quality Attribute Scenarios	Formulating 6-part concrete, testable quality attribute scenarios (Performance, Availability, Security, etc.).	**Nour Maatouk**
6	Constraints	Defining technical, business, and organizational constraints governing system architecture.	**Nour Maatouk**
7	Architecturally Significant Requirements (ASRs)	Ranking top requirements & constructing the Utility Tree / Prioritized ASR analysis.	**Ahmed Alyasergy**